Feed-forward and Convolutional Networks on Fashion-MNIST

Technical report · Week 2, tasks 2.1 – 2.3 · PyTorch 2.14.0 (CPU) · seed 42 · 13 experiment runs, 76 min of training compute

1. Introduction

This report covers three pieces of work on Fashion-MNIST: a feed-forward network built from scratch in PyTorch, a convolutional classifier with data augmentation, and a controlled hyperparameter study on the convolutional model. The final test accuracy of the CNN is **0.9067** against a chance baseline of 0.1000 and **0.8844** for the feed-forward network on the same data.

The headline finding is not that one optimizer won. Across the nine optimizer/learning-rate combinations tested, the top three finished within 0.12 accuracy points of each other — a quarter of the measured noise floor. What the experiments actually separated cleanly was **how badly each optimizer fails** when the learning rate is wrong: across the grid, RMSprop lost 80.1 accuracy points, Adam 10.3, and SGD with momentum 0.45.

2. Setup

Dataset. Fashion-MNIST throughout — 60,000 training and 10,000 test images, 28×28 greyscale, 10 clothing classes. Chosen over MNIST, which saturates near 99% with almost no effort and leaves nothing to learn from the curves. The training set was split 54,000 / 6,000 into train and validation *per class*, so validation keeps the same class balance as training. The official test set was untouched until final evaluation.

Model	Layers	Params	Optimiser	Test acc
Feed-forward (2.1)	784 → 256 → 128 → 10, ReLU, dropout 0.2	235,146	Adam 1e-3, bs 64, 8 ep	**0.8844**
CNN (2.2)	2 × (Conv3×3 + BN + ReLU + MaxPool2) → Flatten → 128 → 10	421,738	Adam 1e-3, wd 1e-4, bs 64, 8 ep	**0.9067**

Both models see identical normalised inputs. The CNN's advantage is structural: its first convolution costs 288 parameters where the MLP's first dense layer costs 200,960, and the weight sharing that buys is what lets it recognise a garment wherever it sits in the frame. Per-class accuracy on the test set ranges from 0.7060 (Shirt) to 0.9900 (Bag), so the network is not uniformly strong — the residual errors concentrate in the Shirt / Pullover / T-shirt/top / Coat cluster, which differ by texture rather than shape. The single largest confusion is Shirt read as T-shirt/top: 122 of the 1,000 Shirt images in the test set.

2.1 An augmentation ablation that did not go the expected way

The 2.2 CNN was trained with RandomCrop(28, padding=2) and RandomHorizontalFlip(0.5) on the training loader only. Switching augmentation off, with the architecture, seed and schedule held fixed, **improved** validation accuracy from 0.8872 to 0.9005 over two epochs — 1.33 points in favour of no augmentation. This runs against the usual expectation, and the likely reason is visible in the training curves: the augmented model had not begun to overfit, so the transforms were spending capacity on an invariance the data did not yet demand. Augmentation was subsequently **disabled for all 13 hyperparameter runs** so that configurations would be compared on the same distribution, and to remove a source of run-to-run variance.

3. Experiment design and results

A full grid of 3 optimisers × 3 learning rates × 3 batch sizes × 3 epoch counts is 81 runs, roughly 36 CPU-hours. Instead a **greedy one-factor-at-a-time search** was used: stage A varies optimiser and learning rate (9 runs, batch 64, 3 epochs); stage B varies batch size at stage A's winner; stage C runs 12 epochs at the combined winner. Inside each stage every row differs from its neighbours in exactly one setting. The design is order-dependent and uses one seed per configuration — limitations noted in section 4.

Table 1 — Complete experiment log. 13 runs, 76 minutes total. Validation accuracy, 6,000 held-out images.

Run	Optimiser	LR	Batch	Ep	Train loss	Val acc	Best val
A-adam-lr0.1-bs64	adam	0.1	64	3	0.7214	**0.7975**	0.7975
A-adam-lr0.01-bs64	adam	0.01	64	3	0.2926	**0.9007**	0.9007
A-adam-lr0.001-bs64	adam	0.001	64	3	0.2440	**0.9002**	0.9002
A-rmsprop-lr0.1-bs64	rmsprop	0.1	64	3	2.3133	**0.1000** †	0.1000
A-rmsprop-lr0.01-bs64	rmsprop	0.01	64	3	0.3177	**0.8660**	0.8817
A-rmsprop-lr0.001-bs64	rmsprop	0.001	64	3	0.2607	**0.9013**	0.9013

Run	Optimiser	LR	Batch	Ep	Train loss	Val acc	Best val
A-sgd-lr0.1-bs64	sgd	0.1	64	3	0.3594	**0.8948**	0.8948
A-sgd-lr0.01-bs64	sgd	0.01	64	3	0.2607	**0.8978**	0.8978
A-sgd-lr0.001-bs64	sgd	0.001	64	3	0.3145	**0.8933**	0.8933
B-rmsprop-lr0.001-bs16	rmsprop	0.001	16	3	0.2547	**0.9030**	0.9030
B-rmsprop-lr0.001-bs32	rmsprop	0.001	32	3	0.2532	**0.8927**	0.8927
B-rmsprop-lr0.001-bs64	rmsprop	0.001	64	3	0.2607	**0.9013**	0.9013
C-rmsprop-lr0.001-bs16	rmsprop	0.001	16	12	0.1520	**0.9150**	0.9218
C (12 ep, best epoch)	rmsprop	0.001	16	9	0.1715	**0.9218**	0.9218

† RMSprop at lr = 0.1 did not diverge to NaN — it converged to a uniform prediction, sitting at exactly chance (0.1000) with training loss 2.3133 against ln(10) = 2.3026. The divergence check in the logging code did not fire, because the loss stayed finite.

4. Key takeaways

1. Optimiser choice mattered less than learning-rate choice. The best three configurations — rmsprop at 0.001, adam at 0.01, and adam at 0.001 — span 0.12 accuracy points, a quarter of the 0.41-point noise floor. They are a three-way tie, and the usual claim that one optimiser is simply better is not visible in this data.

2. Fragility, not peak accuracy, is what separated the optimisers. Across two orders of magnitude of learning rate, SGD with momentum varied by 0.45 points (at the noise floor), Adam by 10.3, and RMSprop by 80.1. RMSprop at lr = 0.1 collapsed completely, because its update size is roughly the learning rate itself, so 0.1 throws the network out of the region where the BatchNorm statistics mean anything. Adam survived the same setting with ten points lost. The practical default carried forward is Adam at 1e-3: not a winner here, but the one that fails locally and visibly rather than silently.

3. Batch size showed no effect this study can defend. Batch sizes 16, 32 and 64 spread 1.03 points — above the noise floor, but not by much, and non-monotonic (0.9030, 0.8927, 0.9013). A genuine batch-size effect has a direction, since smaller batches mean noisier gradients and more implicit regularisation. An interior minimum has no mechanism behind it, so the spread is not attributable to batch size. Ranking batch 16 first would be reading noise.

4. Epoch count was the only setting with a large, unambiguous effect. Going from 3 to 12 epochs gained 1.20 points, roughly three times the noise floor. Validation accuracy peaked at 0.9218 on epoch 9 and then declined slightly, while the train/validation gap kept widening after epoch 5. For this architecture and dataset the useful range is roughly 9–12 epochs, and that is where the compute is worth spending.

5. Measuring the noise floor changed what could be claimed. Epoch-to-epoch jitter on the 12-epoch run, once it had flattened, put the floor at 0.41 accuracy points. Several apparent orderings in the log fall below it. Without that measurement, the batch-size result would have been reported as a finding.

5. Limitations

Three, in rough order of how much they should change the conclusions. **First, one seed per configuration.** The noise floor was inferred from within-run jitter with the weights held fixed, which is a lower bound on true run-to-run variance; a seed sweep would give a larger number and could reclassify some of the 'at the noise floor' results. **Second, the search is greedy.** Batch size was only explored at stage A's winning optimiser, so a different pairing might prefer a different batch size, and the stages are not independent. **Third, the architecture was held fixed throughout**, so nothing here says how these hyperparameters interact with a different model. The wall-clock columns in the raw log are also unreliable: one run recorded 4.2× the per-epoch cost of an identical configuration, and a dedicated timing test failed to reproduce it, so no timing conclusions are drawn.

Natural next step. 2.2 found validation accuracy still rising at epoch 8, and 2.3 confirms the ceiling is reached near epoch 9–12. Extending that run, and re-testing augmentation once the model is actually overfitting, is the highest-value follow-up — it is the one place where a negative result in this report is likely to be a budget artefact rather than a real effect.